

React 18 – Complete Guide (30 LPA MERN Bootcamp)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 14 Sep 2026 – 4 Oct 2026 (Days 29–49)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Target level: Senior frontend interviews at product companies. You must explain hooks internals, reconciliation, performance trade-offs, and React 18 concurrent features — not just syntax.

Table of Contents

1. [What is React?](#)
2. [JSX — Deep Dive](#)
3. [Components & Composition](#)
4. [Props — Patterns & Pitfalls](#)
5. [State — Local, Lifted, Derived](#)
6. [All Hooks In Depth](#)
7. [Forms — Controlled, Uncontrolled, Libraries](#)
8. [Performance Optimization](#)
9. [Routing — React Router v6](#)
10. [State Management](#)
11. [Error Boundaries](#)
12. [React 18 Features](#)
13. [Patterns & Anti-Patterns](#)

14. 15 Interview Q&A

15. Where to Practice

1. What is React?

React is a **declarative UI library** for building component trees. You describe *what* the UI should look like for a given state; React determines *how* to update the DOM efficiently.

Core Concepts

Concept	Description
Components	Reusable UI units (functions returning JSX)
Props	Read-only inputs from parent to child
State	Mutable data owned by a component
Virtual DOM	In-memory tree; React diffs against previous render
Reconciliation	Algorithm comparing trees to compute minimal DOM updates
One-way data flow	Data flows down via props; events flow up via callbacks

React vs Framework

	React	Next.js / Remix
Routing	Bring your own (React Router)	Built-in file-based
SSR	Via libraries	First-class
Data fetching	useEffect / TanStack Query	Server components, loaders

Interview Box: "Is React a framework?"

Answer: No — it's a view library. You choose router, state manager, and data layer. Next.js is a React *framework* that adds opinions.

2. JSX – Deep Dive

JSX is syntactic sugar for `React.createElement(type, props, ...children)`.

2.1 Basic Rules

```
// Expression container
const name = "Priya";
const element = <h1 className="title">Hello, {name.toUpperCase()}</h1>;

// Attributes – camelCase in JS


// Boolean props shorthand
<input disabled />           // disabled={true}
<Modal open={isOpen} />

// Fragments – no extra DOM node
<>
  <Header />
  <Main />
</>

// Keys in lists
{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

2.2 JSX vs HTML Differences

HTML	JSX
<code>class</code>	<code>className</code>
<code>for</code>	<code>htmlFor</code>
<code>onclick</code>	<code>onClick</code>
<code>tabindex</code>	<code>tabIndex</code>
<code>style="color: red"</code>	<code>style={{ color: "red" }}</code>
Self-closing optional	Self-closing required (<code>
</code>)

2.3 Conditional Rendering

```
function StatusBadge({ status }) {  
  // Ternary  
  return status === "active" ? <span className="green">Active</span> : null;  
  
  // Logical AND – careful: {count && <Badge />} renders "0" when count is 0  
  // Fix: {count > 0 && <Badge count={count} />}  
  
  // Early return  
  if (!status) return null;  
  return <span>{status}</span>;  
}
```

2.4 Children Patterns

```
function Layout({ header, sidebar, children }) {  
  return (  
    <div className="layout">  
      <header>{header}</header>  
      <aside>{sidebar}</aside>  
      <main>{children}</main>  
    </div>  
  );  
}  
  
// Render prop  
function MouseTracker({ render }) {  
  const [pos, setPos] = useState({ x: 0, y: 0 });  
  useEffect(() => {  
    const handler = (e) => setPos({ x: e.clientX, y: e.clientY });  
    window.addEventListener("mousemove", handler);  
    return () => window.removeEventListener("mousemove", handler);  
  }, []);  
  return render(pos);  
}
```

3. Components & Composition

3.1 Function Components (Standard)

```
function Welcome({ name, age = 18 }) {  
  return (  
    <div>  
      <h1>Hello, {name}</h1>  
      <p>Age: {age}</p>  
    </div>  
  );  
}
```

3.2 TypeScript Props

```
interface WelcomeProps {  
  name: string;  
  age?: number;  
}  
  
function Welcome({ name, age = 18 }: WelcomeProps) {  
  return <h1>Hello, {name}</h1>;  
}
```

3.3 Composition over Inheritance

```
function Dialog({ title, children, footer }) {
  return (
    <div role="dialog" aria-labelledby="dialog-title">
      <h2 id="dialog-title">{title}</h2>
      <div className="dialog-body">{children}</div>
      {footer} && <div className="dialog-footer">{footer}</div>
    </div>
  );
}

function ConfirmDialog({ message, onConfirm }) {
  return (
    <Dialog
      title="Confirm"
      footer={
        <>
          <button onClick={onConfirm}>Yes</button>
          <button>Cancel</button>
        </>
      }
    >
      <p>{message}</p>
    </Dialog>
  );
}
```

3.4 Compound Components

```
const TabsContext = createContext(null);

function Tabs({ children, defaultTab }) {
  const [active, setActive] = useState(defaultTab);
  return (
    <TabsContext.Provider value={{ active, setActive }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

Tabs.List = function TabList({ children }) {
  return <div role="tablist">{children}</div>;
};

Tabs.Tab = function Tab({ id, children }) {
  const { active, setActive } = useContext(TabsContext);
  return (
    <button
      role="tab"
      aria-selected={active === id}
      onClick={() => setActive(id)}
    >
      {children}
    </button>
  );
};

Tabs.Panel = function TabPanel({ id, children }) {
  const { active } = useContext(TabsContext);
  if (active !== id) return null;
  return <div role="tabpanel">{children}</div>;
};
```

4. Props – Patterns & Pitfalls

4.1 Props Are Read-Only

```
// ❌ Never mutate props
function Bad({ user }) {
  user.name = "hacked"; // Anti-pattern
}

// ✅ Clone or derive
function Good({ user, onUpdate }) {
  return <button onClick={() => onUpdate({ ...user, name: "new" })}>Edit</button>;
}
```

4.2 Prop Drilling vs Context

Approach	When
Props	1-2 levels deep
Context	Theme, auth, locale – many consumers
State manager	Complex shared state with actions

4.3 Spread Props

```
function Input({ label, ...inputProps }) {
  return (
    <label>
      {label}
      <input {...inputProps} />
    </label>
  );
}
```

4.4 Default Props (Modern)

```
// Prefer default parameters over defaultProps (deprecated for function components)
function Avatar({ size = 40, src, alt = "User avatar" }) {
  return <img width={size} height={size} src={src} alt={alt} />;
}
```

5. State – Local, Lifted, Derived

5.1 Local State

```
function Counter() {
  const [count, setCount] = useState(0);

  // Functional update – safe when next state depends on previous
  const increment = () => setCount(c => c + 1);

  return <button onClick={increment}>{count}</button>;
}
```

5.2 Lifted State

When siblings need shared data, lift state to common ancestor.

```
function TemperatureConverter() {  
  const [celsius, setCelsius] = useState("");  
  
  const fahrenheit =  
    celsius === "" ? "" : String((Number(celsius) * 9) / 5 + 32);  
  
  return (  
    <>  
      <CelsiusInput value={celsius} onChange={setCelsius} />  
      <FahrenheitDisplay value={fahrenheit} />  
    </>  
  );  
}
```

5.3 Derived State – Don't Duplicate

```
// ❌ Bad – selectedItem stored AND derived from items  
const [items, setItems] = useState([]);  
const [selectedId, setSelectedId] = useState(null);  
const selected = items.find(i => i.id === selectedId); // ✅ Derive, don't store
```

5.4 State Placement Decision Tree

Is this data needed by multiple unrelated branches?

- ├ Yes → Context / Zustand / Redux
- └ No → Is it needed by siblings?
 - ├ Yes → Lift to parent
 - └ No → Local useState

6. All Hooks In Depth

6.1 Rules of Hooks

1. Only call at **top level** — not in loops, conditions, or nested functions.
2. Only call from **React function components** or **custom hooks**.

React relies on call order to associate state with each hook instance.

6.2 useState

```
const [state, setState] = useState(initialValue);  
const [state, setState] = useState(() => expensiveInit()); // lazy init
```

Pattern	Example
Object state	<code>setUser(u => ({ ...u, name: "x" }))</code>
Array state	<code>setItems(prev => [...prev, newItem])</code>
Reset	<code>setCount(0)</code>

6.3 useEffect

Runs **after paint** (async relative to render).

```
useEffect(() => {  
  // Effect body  
  document.title = `Count: ${count}`;  
  
  return () => {  
    // Cleanup – before re-run or unmount  
  };  
}, [count]); // dependency array
```

Dependency array	Behavior
Omitted	Runs every render (rarely wanted)
<code>[]</code>	Runs once on mount
<code>[a, b]</code>	Runs when a or b change

```
// Fetch pattern
useEffect(() => {
  let cancelled = false;

  async function load() {
    const data = await fetchUser(id);
    if (!cancelled) setUser(data);
  }
  load();

  return () => { cancelled = true; };
}, [id]);
```

Interview Box: "useEffect vs useLayoutEffect?"

Answer: `useLayoutEffect` fires synchronously after DOM mutations, before browser paint. Use for DOM measurements or preventing visual flicker. `useEffect` is default for data fetching and subscriptions.

6.4 useLayoutEffect

```
useLayoutEffect(() => {
  const height = ref.current.getBoundingClientRect().height;
  setTooltipHeight(height);
}, []);
```

6.5 useRef

```
function TextInput() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current?.focus();
  }, []);

  return <input ref={inputRef} />;
}

// Mutable box – doesn't trigger re-render
function Timer() {
  const intervalRef = useRef(null);
  const countRef = useRef(0);

  const start = () => {
    intervalRef.current = setInterval(() => {
      countRef.current += 1;
    }, 1000);
  };

  return <button onClick={start}>Start</button>;
}
```

6.6 useMemo

Memoizes **computed values**.

```
const sortedItems = useMemo(
  () => items.slice().sort((a, b) => a.name.localeCompare(b.name)),
  [items]
);
```

6.7 useCallback

Memoizes **function references** — useful when passing to memoized children.

```
const handleClick = useCallback(() => {  
  doSomething(id);  
}, [id]);  
  
return <MemoizedChild onClick={handleClick} />;
```

Interview Box: "When should you use useMemo/useCallback?"

Answer: When profiling shows a problem, or when a stable reference is required (dependency arrays, React.memo children). Premature memoization adds complexity without benefit.

6.8 useReducer

Better for complex state transitions.

```
const initialState = { items: [], loading: false, error: null };

function reducer(state, action) {
  switch (action.type) {
    case "FETCH_START":
      return { ...state, loading: true, error: null };
    case "FETCH_SUCCESS":
      return { ...state, loading: false, items: action.payload };
    case "FETCH_ERROR":
      return { ...state, loading: false, error: action.payload };
    default:
      return state;
  }
}

function ItemList() {
  const [state, dispatch] = useReducer(reducer, initialState);

  useEffect(() => {
    dispatch({ type: "FETCH_START" });
    fetchItems()
      .then(data => dispatch({ type: "FETCH_SUCCESS", payload: data }))
      .catch(err => dispatch({ type: "FETCH_ERROR", payload: err.message }));
  }, []);

  if (state.loading) return <Spinner />;
  if (state.error) return <Error message={state.error} />;
  return <ul>{state.items.map(i => <li key={i.id}>{i.name}</li>)}</ul>;
}
```

6.9 useContext

```
const ThemeContext = createContext("light");

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");
  const value = useMemo(() => ({ theme, setTheme }), [theme]);
  return (
    <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>
  );
}

function ThemedButton() {
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <button className={theme} onClick={() => setTheme(t => t === "light" ? "dark"
      Toggle
    </button>
  );
}
```

Performance note: All consumers re-render when context value changes. Split contexts or use selectors (Zustand, Jotai).

6.10 useId

Stable IDs for accessibility — SSR-safe.

```
function PasswordField() {
  const id = useId();
  return (
    <>
      <label htmlFor={id}>Password</label>
      <input id={id} type="password" />
    </>
  );
}
```

6.11 useImperativeHandle

Expose limited imperative API via ref (with `forwardRef`).

```
const FancyInput = forwardRef(function FancyInput(props, ref) {
  const inputRef = useRef(null);

  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current?.focus(),
    clear: () => { if (inputRef.current) inputRef.current.value = ""; },
  }));

  return <input ref={inputRef} {...props} />;
});
```

6.12 useDeferredValue & useTransition (React 18)

```
function SearchResults({ query }) {
  const deferredQuery = useDeferredValue(query);
  const isStale = query !== deferredQuery;

  const results = useMemo(
    () => filterHugeList(deferredQuery),
    [deferredQuery]
  );

  return (
    <ul style={{ opacity: isStale ? 0.6 : 1 }}>
      {results.map(r => <li key={r.id}>{r.name}</li>)}
    </ul>
  );
}

function TabContainer() {
  const [tab, setTab] = useState("home");
  const [isPending, startTransition] = useTransition();

  const selectTab = (next) => {
    startTransition(() => setTab(next));
  };

  return (
    <>
      {isPending && <Spinner />}
      <TabBar active={tab} onSelect={selectTab} />
      <TabContent tab={tab} />
    </>
  );
}
```

6.13 useSyncExternalStore

Subscribe to external stores (Redux, Zustand internals, browser APIs).

```
function useOnlineStatus() {  
  return useSyncExternalStore(  
    (callback) => {  
      window.addEventListener("online", callback);  
      window.addEventListener("offline", callback);  
      return () => {  
        window.removeEventListener("online", callback);  
        window.removeEventListener("offline", callback);  
      };  
    },  
    () => navigator.onLine,  
    () => true // server snapshot  
  );  
}
```

6.14 Custom Hooks

```
function useDebounce(value, delayMs = 300) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(id);
  }, [value, delayMs]);

  return debounced;
}

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);

    fetch(url)
      .then(r => r.json())
      .then(json => { if (!cancelled) setData(json); })
      .catch(err => { if (!cancelled) setError(err); })
      .finally(() => { if (!cancelled) setLoading(false); });

    return () => { cancelled = true; };
  }, [url]);

  return { data, loading, error };
}
```

Hooks Quick Reference

Hook	Purpose
<code>useState</code>	Local state
<code>useEffect</code>	Side effects after render
<code>useLayoutEffect</code>	Sync DOM effects before paint
<code>useRef</code>	DOM ref / mutable value
<code>useMemo</code>	Memoize computed value
<code>useCallback</code>	Memoize function
<code>useReducer</code>	Complex state logic
<code>useContext</code>	Consume context
<code>useId</code>	Stable unique IDs
<code>useDeferredValue</code>	Defer expensive renders
<code>useTransition</code>	Mark updates non-urgent
<code>useSyncExternalStore</code>	External store subscription

7. Forms – Controlled, Uncontrolled, Libraries

7.1 Controlled Components

```
function LoginForm() {  
  const [email, setEmail] = useState("");  
  const [password, setPassword] = useState("");  
  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    login({ email, password });  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input  
        type="email"  
        value={email}  
        onChange={(e) => setEmail(e.target.value)}  
        required  
      />  
      <input  
        type="password"  
        value={password}  
        onChange={(e) => setPassword(e.target.value)}  
        required  
      />  
      <button type="submit">Login</button>  
    </form>  
  );  
}
```

7.2 Uncontrolled + useRef

```
function QuickSearch() {  
  const inputRef = useRef(null);  
  
  const handleSearch = () => {  
    const query = inputRef.current?.value;  
    search(query);  
  };  
  
  return (  
    <>  
      <input ref={inputRef} defaultValue="" />  
      <button onClick={handleSearch}>Search</button>  
    </>  
  );  
}
```

7.3 React Hook Form (Production Standard)

```
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

const schema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
});

function RegisterForm() {
  const { register, handleSubmit, formState: { errors, isSubmitting } } = useForm({
    resolver: zodResolver(schema),
  });

  const onSubmit = async (data) => {
    await registerUser(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register("email")} />
      {errors.email && <span>{errors.email.message}</span>}
      <input type="password" {...register("password")} />
      {errors.password && <span>{errors.password.message}</span>}
      <button disabled={isSubmitting}>Register</button>
    </form>
  );
}
```

Approach	Pros	Cons
Controlled	Full control, instant validation	Re-render per keystroke
Uncontrolled	Fewer renders	Harder validation UX
React Hook Form	Performance + validation	Extra dependency

8. Performance Optimization

8.1 React.memo

```
const ExpensiveList = memo(function ExpensiveList({ items, onSelect }) {
  return (
    <ul>
      {items.map(item => (
        <li key={item.id} onClick={() => onSelect(item.id)}>{item.name}</li>
      ))}
    </ul>
  );
});
```

Shallow prop comparison — re-renders only when props change by reference/value.

8.2 Code Splitting

```
import { lazy, Suspense } from "react";

const Dashboard = lazy(() => import("./Dashboard"));

function App() {
  return (
    <Suspense fallback=<PageLoader />>
      <Dashboard />
    </Suspense>
  );
}
```

8.3 Virtualization

For 10k+ row lists, use `@tanstack/react-virtual` or `react-window` — render only visible items.

8.4 Profiling Checklist

Symptom	Fix
Slow initial load	Code split, tree-shake
Slow list scroll	Virtualize
Child re-renders on parent	<code>memo</code> + stable callbacks
Expensive computation	<code>useMemo</code>
Large context	Split context / selectors

8.5 Reconciliation & Keys

```
// ❌ Index as key when list reorders
{items.map((item, index) => <Row key={index} item={item} />)}

// ✅ Stable unique ID
{items.map(item => <Row key={item.id} item={item} />)}
```

Wrong keys cause incorrect state preservation and poor performance.

9. Routing – React Router v6

```
import {
  createBrowserRouter,
  RouterProvider,
  Route,
  Link,
  Outlet,
  useParams,
  useNavigate,
  Navigate,
} from "react-router-dom";

const router = createBrowserRouter([
  {
    path: "/",
    element: <RootLayout />,
    errorElement: <ErrorPage />,
    children: [
      { index: true, element: <Home /> },
      { path: "users/:userId", element: <UserProfile /> },
      {
        path: "dashboard",
        element: <ProtectedRoute><Dashboard /></ProtectedRoute>,
      },
    ],
  },
]);

function RootLayout() {
  return (
    <>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/dashboard">Dashboard</Link>
      </nav>
      <Outlet />
    </>
  );
}

function UserProfile() {
  const { userId } = useParams();
  const navigate = useNavigate();
  return (
    <div>
      <h1>User {userId}</h1>
      <button onClick={() => navigate(-1)}>Back</button>
    </div>
  );
}
```

```
}  
  
function App() {  
  return <RouterProvider router={router} />;  
}
```

Data Loading (v6.4+ loaders)

```
{  
  path: "users/:id",  
  element: <User />,  
  loader: async ({ params }) => {  
    return fetchUser(params.id);  
  },  
}  
  
function User() {  
  const user = useLoaderData();  
  return <h1>{user.name}</h1>;  
}
```

10. State Management

Decision Matrix

Solution	Best for
<code>useState</code> / <code>useReducer</code>	Component-local
Context	Theme, auth, low-frequency global
Zustand	Medium apps – minimal boilerplate
Redux Toolkit	Large teams, time-travel, middleware
TanStack Query	Server state (API cache)
Jotai / Recoil	Atomic fine-grained updates

Zustand Example

```
import { create } from "zustand";

const useCartStore = create((set) => ({
  items: [],
  addItem: (product) =>
    set((state) => ({ items: [...state.items, product] })),
  removeItem: (id) =>
    set((state) => ({ items: state.items.filter(i => i.id !== id) })),
  total: () => useCartStore.getState().items.reduce((s, i) => s + i.price, 0),
}));
```

TanStack Query – Server State

```
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";

function UserList() {
  const { data, isLoading, error } = useQuery({
    queryKey: ["users"],
    queryFn: fetchUsers,
    staleTime: 60_000,
  });

  if (isLoading) return <Spinner />;
  if (error) return <Error message={error.message} />;

  return (
    <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>
  );
}
```

Interview Box: "Client state vs server state?"

Answer: Client state lives in UI (modal open, form draft). Server state is fetched, cached, and can be stale. Don't put API data in Redux if TanStack Query handles cache, refetch, and deduplication.

11. Error Boundaries

Class components only (no hook equivalent yet).

```
class ErrorBoundary extends React.Component {
  state = { hasError: false, error: null };

  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  componentDidCatch(error, info) {
    logToService(error, info.componentStack);
  }

  render() {
    if (this.state.hasError) {
      return this.props.fallback ?? <h1>Something went wrong.</h1>;
    }
    return this.props.children;
  }
}

// Usage
<ErrorBoundary fallback={<ErrorPage />}>
  <AppRoutes />
</ErrorBoundary>
```

Limitations: Error boundaries don't catch event handler errors, async errors, or SSR errors — use try/catch there.

```
async function handleSubmit() {
  try {
    await saveData();
  } catch (err) {
    setError(err.message);
  }
}
```

12. React 18 Features

12.1 Automatic Batching

Multiple state updates in event handlers, promises, and timeouts batch into one re-render.

```
// React 18 – one re-render
setTimeout(() => {
  setCount(c => c + 1);
  setFlag(f => !f);
}, 1000);
```

12.2 Concurrent Rendering

React can interrupt low-priority renders. Enables `useTransition`, `useDeferredValue`, and Suspense improvements.

12.3 Strict Mode (Development)

Double-invokes effects and renders to surface side-effect bugs.

```
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

12.4 New Root API

```
import { createRoot } from "react-dom/client";

const root = createRoot(document.getElementById("root"));
root.render(<App />);
```

12.5 Suspense Improvements

SSR streaming, selective hydration — used heavily in Next.js App Router.

Feature	Benefit
<code>useTransition</code>	Keep UI responsive during heavy updates
<code>useDeferredValue</code>	Lag behind fast input for expensive derived UI
Streaming SSR	Send HTML shell first, stream components
Automatic batching	Fewer renders, better perf

13. Patterns & Anti-Patterns

✓ Recommended Patterns

Pattern	Use case
Container/Presentational	Separate data fetching from UI
Custom hooks	Reuse stateful logic
Compound components	Flexible APIs (Tabs, Select)
Lifting state up	Shared sibling state
Render props / children fn	Inject behavior
Error boundaries	Graceful failure UI
Suspense + lazy	Route-level code splitting

✖ Anti-Patterns

Anti-pattern	Why it's bad	Fix
Mutating state	No re-render, broken UI	Spread / immutable updates
Index as key (dynamic lists)	Wrong component state	Stable unique keys
useEffect for everything	Race conditions, stale closures	Event handlers, TanStack Query
Prop drilling 5+ levels	Unmaintainable	Context / store
Giant components	Untestable	Split by responsibility
Derived state in useEffect	Extra render cycle	Compute during render
<code>{count && <X />}</code> when count can be 0	Renders "0"	<code>count > 0 &&</code>
Missing cleanup in useEffect	Memory leaks	Return cleanup function

```
// ✖ Syncing props to state – usually wrong
function Bad({ userId }) {
  const [user, setUser] = useState(null);
  useEffect(() => {
    fetchUser(userId).then(setUser);
  }, [userId]);
  // Fine for fetch – but NOT: useEffect(() => setName(propName), [propName])
}
```

14. 15 Interview Q&A

Q1: Virtual DOM — how does reconciliation work?

React builds a fiber tree. On state change, it creates a new tree, diffs with previous (same type → update props; different type → replace subtree), batches DOM writes.

Q2: Why must keys be stable and unique?

Keys help React match list items across renders. Unstable keys cause remounts, lost local state, and incorrect animations.

Q3: Controlled vs uncontrolled components?

Controlled: React state is source of truth (`value` + `onChange`). Uncontrolled: DOM holds state (`defaultValue` , refs).

Q4: Explain the `useEffect` dependency array.

ESLint `exhaustive-deps` ensures values used inside the effect are listed. Missing deps → stale closures; over-listing → extra runs.

Q5: What causes unnecessary re-renders?

Parent re-render, new object/function props each render, context changes. Fix with memoization, composition, or state colocation.

Q6: `useMemo` vs `useCallback`?

`useMemo` caches values; `useCallback` caches functions. `useCallback(fn, deps) ≡ useMemo(() => fn, deps)`.

Q7: When to use `useReducer` over `useState`?

Multiple related state fields, complex transitions, next state depends on previous in non-trivial ways, or testing reducer in isolation.

Q8: What is prop drilling and how do you fix it?

Passing props through many intermediate components. Fix with Context, component composition (children), or a state library.

Q9: Error boundaries — what do they NOT catch?

Errors in event handlers, async code (setTimeout, promises), SSR, or errors in the boundary itself.

Q10: React 18 automatic batching?

All updates in the same event loop tick batch — including promises and native event handlers — reducing re-renders.

Q11: `useTransition` vs `useDeferredValue`?

`useTransition` wraps state updates you initiate. `useDeferredValue` defers a value you're consuming (often from props/state).

Q12: How does Strict Mode help?

Double-mounts components in dev to expose missing effect cleanups and non-idempotent side effects.

Q13: Why prefer TanStack Query for API data?

Built-in caching, deduplication, background refetch, stale-while-revalidate, loading/error states — avoids reinventing in `useEffect`.

Q14: `forwardRef` — when needed?

When parent needs DOM ref to child internal element (focus, measure). Function components don't expose refs by default.

Q15: How would you optimize a 10,000-item list?

Virtualization, pagination, memoized row components, stable keys, avoid inline functions in render if rows are memoized, Web Workers for filtering if CPU-bound.

15. Where to Practice

Official Documentation

Resource	URL	Notes
React Docs (current)	react.dev	Canonical – hooks, thinking in React
Legacy beta docs	beta.reactjs.org	Redirects to react.dev; some old links
React Blog	react.dev/blog	React 18, Server Components announcements

Hands-On Projects

Platform	URL	Focus
Frontend Mentor	frontendmentor.io	Real-world UI challenges with designs
React Challenges	github.com/alexgurr/react-coding-challenges	Hooks, patterns
Epic React (Kent C. Dodds)	epicreact.dev	Deep hooks mastery (paid)

Weekly Practice Plan (30 LPA Track)

Week	Focus	Deliverable
1	JSX, components, props	Todo app with TypeScript
2	useState, useEffect, forms	Weather app with API
3	useContext, useReducer, custom hooks	Theme + auth context
4	React Router, code splitting	Multi-page dashboard
5	Performance, memo, virtualization	Optimize 1 Frontend Mentor project
6	TanStack Query + Zustand	E-commerce product listing
7	Error boundaries, testing (RTL)	Production-grade MERN frontend module

Interview Simulation Exercises

1. Build a debounced search with abort controller and loading states.
2. Implement infinite scroll with Intersection Observer + TanStack Query.
3. Create accessible modal (focus trap, ESC close, `useId`).
4. Explain fiber architecture and concurrent rendering to a mock interviewer.
5. Refactor a class component codebase to hooks (if given legacy code).

Recommended Reading Order on react.dev

1. Quick Start → Installation
2. Describing the UI (JSX, components, props)
3. Adding Interactivity (state, events, forms)
4. Managing State (lifting, reducer, context)
5. Escape Hatches (refs, effects, context pitfalls)
6. Hooks API Reference — read every hook once

Previous in bootcamp: [02-typescript-complete.md](#) — TypeScript fundamentals for React.

Next: Node.js & Express — backend layer of MERN stack.

Product Company Interview Bank – React (100 Questions)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Target companies: Google, Meta, Microsoft, Amazon, Adobe, Atlassian, Uber, Flipkart **Rule:** Full terms only — spell out JWT as JSON Web Token, API as Application Programming Interface, etc.

Question 1: What is React and why do companies use it?

Answer: React is a JavaScript library for building user interfaces, created by Meta (formerly Facebook). Companies use it because it lets teams break the user interface into reusable pieces called components, updates the screen efficiently, and has a huge ecosystem of tools and developers. React works well for single-page applications where the page updates without a full reload, which is common in products at Google, Amazon, and Flipkart.

Question 2: What is JSX and how does it relate to JavaScript?

Answer: JSX (JavaScript XML) is a syntax extension that lets you write HTML-like markup inside JavaScript. It is not understood by browsers directly — a tool like Babel converts JSX into regular JavaScript function calls such as `React.createElement`. JSX makes user interface code easier to read because structure and logic live in the same file.

Question 3: What is the Virtual Document Object Model (Virtual DOM)?

Answer: The Virtual Document Object Model is a lightweight copy of the real Document Object Model (DOM) kept in memory. When state changes, React builds a new virtual tree, compares it with the previous one (a process called diffing), and updates only the parts of the real DOM that actually changed. This is faster than rewriting the entire page on every update.

Question 4: What is the difference between a component and an element in React?

Answer: An element is a plain object that describes what you want to see on the screen — for example, a button with certain text. A component is a function or class that returns elements. Components are reusable: you can use `<Button />` many times with different properties (props) instead of repeating the same element description.

Question 5: What are props in React?

Answer: Props (short for properties) are read-only inputs passed from a parent component to a child component. They let you customize how a reusable component behaves or looks. For example, a `<Avatar name="Priya" size="large" />` passes the name and size as props. Props flow one way — from parent to child — which makes data flow predictable.

Question 6: What is state in React?

Answer: State is data that belongs to a component and can change over time. When state changes, React re-renders the component to reflect the new data. Examples include whether a modal is open, the text in an input field, or a list of items loaded from an Application Programming Interface (API). State is private to the component that owns it (unless lifted up to a parent).

Question 7: What is the difference between props and state?

Answer: Props are passed in from outside and should not be changed by the child. State is owned and managed inside the component. Think of props as function arguments (fixed for that render) and state as local variables that can change. Both trigger a re-render when they change, but only state can be updated by the component itself using a setter function.

Question 8: What is the `useState` hook?

Answer: `useState` is a React Hook that adds state to functional components. It returns an array with two items: the current state value and a function to update it. Example: `const [count, setCount] = useState(0)`. When you call `setCount(1)`, React schedules a re-render with the new value. Hooks must be called at the top level of a component, not inside loops or conditions.

Question 9: What is the `useEffect` hook?

Answer: `useEffect` runs side effects after React paints the screen. Side effects include fetching data from an API, setting up subscriptions, or manually changing the Document Object Model. You pass a function and optionally a dependency array. If the dependency array is empty `[]`, the effect runs once after the first render (like `componentDidMount`). If you list values like `[userId]`, the effect re-runs when those values change.

Question 10: What is the dependency array in `useEffect` ?

Answer: The dependency array is the second argument to `useEffect`. It tells React which values the effect depends on. React compares these values between renders; if any changed, the effect runs again. Omitting the array runs the effect after every render. An empty array `[]` runs it only once on mount. Listing `[searchTerm]` re-runs when `searchTerm` changes — useful for refetching search results.

Question 11: How do you clean up effects in `useEffect` ?

Answer: Return a cleanup function from inside `useEffect`. React calls this cleanup before the effect runs again and when the component unmounts. Common uses: cancel an API request, clear a timer, or remove an event listener. Example: `return () => clearInterval(timerId)`. Forgetting cleanup can cause memory leaks or bugs like updating state on an unmounted component.

Question 12: What is the `useRef` hook?

Answer: `useRef` creates a mutable object `{ current: value }` that persists across renders without causing a re-render when it changes. Uses include: holding a reference to a DOM element (`inputRef.current.focus()`), storing a previous value, or keeping a timer ID. Unlike state, updating `ref.current` does not trigger a re-render.

Question 13: What is the `useContext` hook?

Answer: `useContext` lets a component read a value from React Context without passing props through every intermediate layer (prop drilling). You create a Context with `React.createContext`, wrap part of the tree in a Provider, and consume the value with `useContext(MyContext)`. Useful for themes, logged-in user info, or language settings shared across many components.

Question 14: What is the `useReducer` hook?

Answer: `useReducer` manages complex state with a reducer function `(state, action) => newState`, similar to Redux. It returns `[state, dispatch]`. You call `dispatch({ type: 'increment' })` and the reducer computes the next state. Prefer `useReducer` when state logic is complex, involves multiple sub-values, or the next state depends on the previous one.

Question 15: What are `useMemo` and `useCallback` ?

Answer: `useMemo` memoizes (caches) the result of an expensive calculation and only recomputes when dependencies change: `useMemo(() => expensive(a, b), [a, b])`. `useCallback` memoizes a function reference: `useCallback(() => doSomething(a), [a])`. Both help performance by avoiding unnecessary work or preventing child components from re-rendering when a new function reference would otherwise break `React.memo` optimization.

Question 16: What is `React.memo`?

Answer: `React.memo` is a higher-order component that wraps a functional component and skips re-rendering if props have not changed (shallow comparison). Use it for expensive pure components that receive the same props often. It works best together with stable callback props created via `useCallback`. Do not wrap every component — memoization itself has a small cost.

Question 17: What is lifting state up?

Answer: Lifting state up means moving shared state from child components to their closest common parent. The parent owns the state and passes it down as props, along with callback functions for children to request updates. This keeps a single source of truth — for example, two sibling inputs both controlled by one parent `formData` object.

Question 18: What are controlled vs uncontrolled components?

Answer: A controlled component gets its value from React state and updates via `onChange` — React is the single source of truth for the input value. An uncontrolled component stores its own value in the DOM; you read it with a ref when needed (e.g., on form submit). Controlled inputs are preferred for validation and instant feedback; uncontrolled can be simpler for one-off forms.

Question 19: What are keys in lists and why do they matter?

Answer: When rendering a list with `.map()`, each item needs a unique `key` prop. Keys help React identify which items changed, were added, or removed during reconciliation. Use stable unique identifiers like database IDs, not array indexes (unless the list is static and never reordered). Wrong keys cause bugs like wrong input values sticking to the wrong row.

Question 20: What is reconciliation?

Answer: Reconciliation is React's process of comparing the new virtual tree with the previous one and deciding the minimum DOM updates needed. React assumes elements of the same type update in place, and different types tear down the old tree and build a new one. Keys help match list items correctly across renders. Understanding reconciliation explains why you should not mutate state directly.

Question 21: Why should you not mutate state directly?

Answer: React compares state by reference to decide whether to re-render. If you mutate an array or object in place (`items.push(newItem)`), the reference stays the same and React may skip the update or behave unpredictably. Always create a new copy: `setItems([...items, newItem])` or `setUser({ ...user, name: 'New' })`. Immutability also makes change detection and debugging easier.

Question 22: What is a custom hook?

Answer: A custom hook is a JavaScript function whose name starts with `use` and that calls other hooks inside. It lets you extract and reuse stateful logic across components — for example, `useFetch(url)` or `useLocalStorage(key)`. Custom hooks share logic, not state; each component call gets its own independent state.

Question 23: What are the Rules of Hooks?

Answer: (1) Only call hooks at the top level — not inside loops, conditions, or nested functions. (2) Only call hooks from React function components or custom hooks. These rules ensure hooks run in the same order every render so React can match state to the correct hook call. Breaking them causes "Rendered more hooks than during the previous render" errors.

Question 24: What is an Error Boundary?

Answer: An Error Boundary is a class component (or library wrapper) that catches JavaScript errors in its child tree during rendering, logs them, and shows a fallback user interface instead of crashing the whole app. Error Boundaries do not catch errors in event handlers, async code, or server-side rendering. React 19 improves error handling with better recovery options.

Question 25: What is code splitting and `React.lazy` ?

Answer: Code splitting breaks your JavaScript bundle into smaller chunks loaded on demand, reducing initial load time. `React.lazy(() => import('./HeavyComponent'))` loads a component only when it is first rendered. Wrap lazy components in `<Suspense fallback={}<Loading />>` to show a placeholder while the chunk downloads. Common for routes or heavy modals.

Question 26: What is `Suspense` in React?

Answer: `Suspense` lets you display a fallback user interface while child components are loading — typically lazy-loaded components or data-fetching integrations. You wrap components in `<Suspense fallback={}<Spinner />>`. Multiple Suspense boundaries can nest for granular loading states. React 19 expands Suspense support for server-side streaming and data fetching.

Question 27: What is the difference between functional and class components?

Answer: Class components use `class extends React.Component`, `this.state`, and lifecycle methods like `componentDidMount`. Functional components are plain functions using hooks for state and effects. Since hooks (React 16.8+), functional components are the standard — they are shorter, easier to test, and avoid `this` binding confusion. Class components still appear in older codebases.

Question 28: What were class component lifecycle methods?

Answer: Key lifecycle methods: `componentDidMount` (after first render — fetch data), `componentDidUpdate` (after props/state change), `componentWillUnmount` (cleanup before removal). In modern React, `useEffect` replaces these patterns: mount = `useEffect(fn, [])`, update = `useEffect(fn, [deps])`, unmount = return cleanup from `useEffect`. Knowing lifecycle names helps in legacy interviews.

Question 29: What is prop drilling and how do you avoid it?

Answer: Prop drilling is passing props through many intermediate components that do not use them, just to reach a deep child. Solutions: React Context (`useContext`), state management libraries (Redux, Zustand), or component composition (pass components as `children` or render props). Choose Context for low-frequency global data; libraries for complex shared state.

Question 30: What is composition in React?

Answer: Composition means building complex user interfaces by combining simpler components rather than inheritance. React favors "has-a" over "is-a" — e.g., a `Dialog` accepts `children` for flexible content instead of subclassing `Dialog` for every variant. The `children` prop, render props, and slots patterns all support composition.

Question 31: What is a Higher-Order Component (HOC)?

Answer: A Higher-Order Component is a function that takes a component and returns a new component with extra props or behavior — e.g., `withAuth(ProfilePage)` adds authentication checks. HOCs were popular before hooks for reusing logic. Custom hooks often replace HOCs today, but you may still see HOCs in libraries like `connect()` in React-Redux.

Question 32: What is the render props pattern?

Answer: Render props is a pattern where a component receives a function as a prop (often called `render` or `children`) and calls it with data: `<DataFetcher render={({data}) => <List items={data} /> } />`. It shares logic between components. Custom hooks largely supersede render props, but the pattern appears in libraries like React Router and Downshift.

Question 33: What causes unnecessary re-renders and how do you fix them?

Answer: Common causes: state changing high in the tree (all children re-render), new object/function references passed as props each render, and Context value changes affecting all consumers. Fixes: split state locally, `React.memo`, `useMemo` / `useCallback`, split Context into smaller contexts, and virtualize long lists. Profile with React DevTools Profiler before optimizing.

Question 34: What is React Strict Mode?

Answer: `<StrictMode>` is a development-only wrapper that intentionally double-invokes certain functions (render, effects) to surface side effects and unsafe patterns. It warns about deprecated APIs like legacy string refs. It does not affect production builds. Enable it in development to catch bugs early, especially missing effect cleanups.

Question 35: What is the React Fiber architecture?

Answer: Fiber is React's internal reconciliation engine (since React 16). It breaks rendering work into small units that can be paused, prioritized, and resumed — enabling concurrent features. Fiber maintains a linked tree of work units, supports incremental rendering, and powers features like Suspense and transitions. You rarely interact with Fiber directly, but it explains how React stays responsive during heavy updates.

Question 36: What are React Server Components?

Answer: React Server Components (RSC) run on the server and send serialized component output to the client — not JavaScript for that component. They can access databases and file systems directly without exposing secrets to the browser. They do not use hooks like `useState` or `useEffect`. Frameworks like Next.js App Router use Server Components by default. Client Components (marked with `'use client'`) handle interactivity.

Question 37: What is new in React 19?

Answer: React 19 (2024) introduces: Actions (async transitions for forms and mutations), `useActionState` and `useFormStatus` for form handling, document metadata support (`<title>`, `<meta>` in components), ref as a prop (no more `forwardRef` needed), improved Suspense and hydration error messages, and the React Compiler (optional) that auto-memoizes components. It also deprecates some legacy APIs like `defaultProps` on function components.

Question 38: What are Actions in React 19?

Answer: Actions are functions passed to forms or transitions that handle async work (like submitting to a server) with built-in pending states. When you use `<form action={submitAction}>`, React manages loading state and can reset the form on success. Actions integrate with `useTransition` and reduce boilerplate for optimistic updates and error handling in forms.

Question 39: What is `useTransition` in React?

Answer: `useTransition` marks state updates as non-urgent transitions. It returns `[isPending, startTransition]`. Wrap slow updates (like filtering a huge list) in `startTransition(() => setFilter(value))` so typing in a search box stays responsive while the heavy re-render happens in the background. Part of React's concurrent rendering model.

Question 40: What is `useDeferredValue` ?

Answer: `useDeferredValue` delays updating a value until urgent updates finish — similar to debouncing but integrated with React's scheduler. Example: `const deferredQuery = useDeferredValue(query)` — the list filters on `deferredQuery` while the input shows `query` immediately. Useful for keeping input responsive while derived UI catches up.

Question 41: What is hydration in React?

Answer: Hydration is when React attaches event listeners and state to server-rendered HTML on the client. The server sends pre-built HTML for fast first paint; client JavaScript "hydrates" it to make it interactive. Hydration mismatches occur when server and client output differ (e.g., using `Date.now()` during render), causing errors. React 19 improves mismatch recovery.

Question 42: What is the React Compiler?

Answer: The React Compiler (formerly "React Forget") automatically adds memoization at build time so developers write plain JavaScript without manually adding `useMemo`, `useCallback`, and `React.memo`. It analyzes component purity and optimizes re-renders. It is optional and used at Meta in production; adoption is growing for performance-sensitive apps.

Question 43: How do you handle forms in React?

Answer: Options: (1) Controlled components with `useState` per field — full control, good for validation. (2) Uncontrolled with refs — simpler for basic forms. (3) Libraries like React Hook Form or Formik — less boilerplate, built-in validation. (4) React 19 Actions — native async form handling with pending states. Choose based on form complexity and validation needs.

Question 44: How do you fetch data in React?

Answer: Common patterns: `useEffect` + `fetch` with loading/error state; data-fetching libraries (TanStack Query, SWR) for caching, refetching, and deduplication; Server Components in Next.js for server-side fetch; React 19 Actions for mutations. Libraries are preferred in client apps because they handle race conditions, caching, and stale data better than raw `useEffect`.

Question 45: What is TanStack Query (React Query) and why use it?

Answer: TanStack Query is a data-fetching library that manages server state in React apps. It provides caching, background refetching, pagination, optimistic updates, and deduplication of requests. Instead of storing API data in Redux, you declare queries like `useQuery({ queryKey: ['users'], queryFn: fetchUsers })` and the library handles loading, error, and cache states.

Question 46: What is Redux and when do you still need it?

Answer: Redux is a predictable state container for JavaScript apps. It stores global state in a single store, updates via pure reducer functions, and uses actions to describe changes. Use Redux (or Redux Toolkit) when many components share complex client state, you need time-travel debugging, or middleware for logging and async flows. For server state, prefer TanStack Query; for simpler global state, Context or Zustand may suffice.

Question 47: What is React Testing Library?

Answer: React Testing Library helps you test components the way users interact with them — by labels, roles, and text — rather than implementation details. Example: `screen.getByRole('button', { name: /submit/i })`. It encourages accessible markup and pairs with Jest or Vitest. Avoid testing internal state; test visible behavior and user flows.

Question 48: What are common React performance patterns?

Answer: (1) Virtualize long lists (react-window). (2) Lazy-load routes and heavy components. (3) Memoize expensive calculations and callbacks. (4) Avoid inline object/array literals in props when children are memoized. (5) Split Context to limit re-render scope. (6) Debounce expensive search filters. (7) Use the Profiler to measure before optimizing. Premature optimization wastes time — profile first.

Question 49: What is the difference between React and ReactDOM?

Answer: `react` is the core library — components, hooks, state, and reconciliation. `react-dom` is the renderer that updates the browser DOM (`createRoot` , `render` , `hydrateRoot`). Other renderers exist: `react-native` for mobile, `@react-three/fiber` for 3D. In most web interviews, "React" means the core plus `react-dom` together.

Question 50: How would you explain your React project in an interview?

Answer: Structure your answer: (1) Problem – what the app does and for whom. (2) Architecture – component structure, state management choice (Context, Redux, TanStack Query), routing. (3) Technical decisions – why Server Components vs Client Components, performance optimizations you applied. (4) Challenges – e.g., fixing re-render issues, handling auth, or improving load time with code splitting. (5) Impact – metrics

like faster load, fewer bugs, or better user experience. Use concrete examples from your MERN project.

Questions 51-100 (top product companies)

Question 51: What is the rules-of-hooks and why do they exist?

Answer: Hooks must be top-level and same order every render so React can match state by call index. Conditional hooks break that mapping. Meta / Flipkart.

Question 52: useMemo vs useCallback vs React.memo – pick one sentence each.

Answer: useMemo caches a value. useCallback caches a function identity. memo skips a child re-render if props are shallow-equal. Do not use all three by default. Meta.

Question 53: How do you fetch data in React 18 without a waterfall?

Answer: Lift fetch to a parent or use a cache (React Query). In Next, fetch in Server Components. Avoid useEffect-only fetch on every leaf. Adobe / Meta.

Question 54: What is a stale closure in useEffect?

Answer: The effect captured old state because deps were wrong. Fix deps or use a ref for the latest value. Uber / Meta debugging.

Question 55: How do you share state between sibling routes?

Answer: URL search params, a store, or lift to a layout. Context if slow-changing. Atlassian.

Question 56: Explain concurrent rendering in one minute.

Answer: React can interrupt a render to stay responsive (`startTransition`). Urgent updates (typing) win over non-urgent (list filter). Meta.

Question 57: What does `startTransition` do?

Answer: Marks a state update as non-urgent so input stays snappy. Use for big list filters. Meta / Adobe.

Question 58: How do you test a component that fetches?

Answer: Mock fetch / MSW, render, wait for text, assert. RTL `findBy`. Do not test implementation details. Atlassian / Uber.

Question 59: Why is putting an object in `useEffect` deps a footgun?

Answer: New object identity every render → infinite loop. Depend on primitive fields or memoize. Amazon FE.

Question 60: How do you avoid layout thrashing in a list?

Answer: Virtualize, avoid reading layout then writing in a loop, use CSS for animation. Google FE.

Question 61: What is lifting state up – when is it too far?

Answer: Share state in the nearest common parent. Too far = huge re-renders and prop drilling. Flipkart.

Question 62: Controlled form libraries vs raw state – when?

Answer: Raw for 2–3 fields. RHF/Formik for validation schemas and field arrays. Razorpay / Adobe.

Question 63: How do you handle optimistic UI?

Answer: Update local state first, rollback on error, reconcile with server id. Meta / Uber.

Question 64: What is the difference between useRef and useState?

Answer: Ref changes do not re-render. Use for timers, previous values, DOM nodes. All FE.

Question 65: How do portals help modals?

Answer: Render into document.body to escape overflow/z-index. Still need focus trap and Escape. Atlassian ally.

Question 66: Explain React Query staleTime vs cacheTime.

Answer: staleTime: how long data is fresh (no refetch). cacheTime/gcTime: how long unused data stays in memory. Uber / Adobe.

Question 67: How do you prevent XSS in React?

Answer: React escapes text children. Danger is `dangerouslySetInnerHTML` and `href={user}`. Sanitize. All.

Question 68: What is a compound component pattern?

Answer: Parent + children that share implicit state (`Select` + `Option`). Used in design systems. Atlassian.

Question 69: How do you code-split a route?

Answer: `lazy` + `Suspense` or Next dynamic import. Show a skeleton. Google / Adobe.

Question 70: Why did my useEffect run twice in Strict Mode?

Answer: React 18 Strict remounts to find unsafe effects. Cleanup must undo the setup. Meta.

Question 71: How do you type event handlers in TypeScript + React?

Answer: `React.ChangeEvent<HTMLInputElement>`, `FormEvent`. Do not use `any`. Atlassian.

Question 72: What is derived state and why is it a bug source?

Answer: Copying props into state and not syncing. Compute during render instead. Flipkart / Meta.

Question 73: How do you implement tabs accessibly?

Answer: roles tablist/tab/tabpanel, arrows, aria-selected, focus management. Atlassian.

Question 74: Explain the render-props vs hooks history.

Answer: Render props and HOCs were reuse patterns; hooks replaced most of them. Still read old codebases. Adobe.

Question 75: How do you debounce an input in React correctly?

Answer: Debounce the callback or the value (useDebounce), not the component function. Abort fetch. Uber.

Question 76: What is used for?

Answer: Stable unique ids for label/input SSR. Avoid Math.random in render. Next / Adobe.

Question 77: How do you reset state when a user id changes?

Answer: `key={userId}` on the subtree or reset in an effect with that dep. Meta.

Question 78: What belongs in context vs Redux vs component state?

Answer: Local UI → state. App-wide auth/theme → context or store. High-frequency → store with selectors. All senior FE.

Question 79: How do you profile a slow React page?

Answer: React Profiler, why-did-you-render, Chrome performance. Fix the hottest commit. Google FE.

Question 80: What is an error boundary fallback that is hire-level?

Answer: Apologize, retry button, report id, do not blank the whole app. Atlassian.

Question 81: How do you implement a11y for a custom dropdown?

Answer: button trigger, listbox, aria-activedescendant, typeahead optional. Same as autocomplete. Atlassian / Uber.

Question 82: Why is index as key bad in a sortable list?

Answer: State attaches to position, not identity — checkboxes jump. Use ids. All.

Question 83: How do Server Actions interact with useFormStatus?

Answer: Pending state without local useState. Still authenticate on the server. Adobe / Next.

Question 84: What is hydration mismatch from Date?

Answer: Server time $\neq$ client time. Render a placeholder, set time in useEffect. Next.

Question 85: How do you keep a WebSocket in React?

Answer: One connection in useEffect, cleanup close, reconnect with backoff, put messages in state or a store. Uber / Meta.

Question 86: Explain memo of a list item vs virtualizing.

Answer: Memo helps if parents re-render often. Virtualizing helps if you mount thousands of rows. Different problems. Meta.

Question 87: How do you handle file upload progress?

Answer: XMLHttpRequest or fetch + ReadableStream; progress state; abort. Flipkart / Adobe.

Question 88: What is the children prop used for composition?

Answer: Pass nodes instead of config flags — `Modal` footer as children. Atlassian design system.

Question 89: How do you prevent a flash of wrong theme?

Answer: Read theme before paint (inline script) or class on html from cookie. Adobe / Meta.

Question 90: What is useSyncExternalStore?

Answer: Subscribe to an external store safely with concurrent React. Used by state libraries. Meta-adjacent.

Question 91: How do you test keyboard-only flows?

Answer: `userEvent.tab` / arrows, `assert focus`. Required at Atlassian.

Question 92: When is CSS-in-JS a liability?

Answer: Runtime cost, FOUC, harder CSP. CSS modules / Tailwind often enough. Google FE.

Question 93: How do you implement optimistic list add with rollback?

Answer: Temp id, replace on server response, remove on error. Meta.

Question 94: What is the difference between Link prefetch in Next and React Router?

Answer: Next can prefetch the RSC payload on viewport. React Router prefetch is opt-in. Adobe.

Question 95: How do you avoid waterfalls in `useEffect` chains?

Answer: Fetch in parallel (`Promise.all`) or one endpoint that joins. Amazon FE.

Question 96: What makes a design-system Button hire-level?

Answer: variants, disabled, loading, aria, `asChild/slot`, no extra wrappers breaking flex. Atlassian.

Question 97: How do you handle multi-step forms?

Answer: Lift step state, persist in URL or session, validate per step. Razorpay / Adobe.

Question 98: Explain why you should not fetch in the constructor or render.

Answer: Side effects belong in effects or server loaders. Render must be pure. All.

Question 99: How do you cancel React Query requests on key change?

Answer: queryKey includes the id; the library aborts/ignores stale. Uber.

Question 100: Walk through your React project like an interviewer is hostile.

Answer: Problem, users, metric, component tree, hardest bug, what you would rewrite. Phase 8A. All companies.